

BACKEND PROBLEM SOLVING

POINT PAYMENT LAB

기존 API 흐름의 문제를 재현하고
요청·응답·DB 상태가 왜 달라졌는지 증명한 결제 개선 프로젝트

기존 문제점 → 수정한 부분 → 개선 결과 → 달라진 이유

Java 17 · Spring Boot · JPA · MySQL · Redis · Redisson

이소윤 · 개인 프로젝트 · 백엔드 설계 / 구현 / 검증

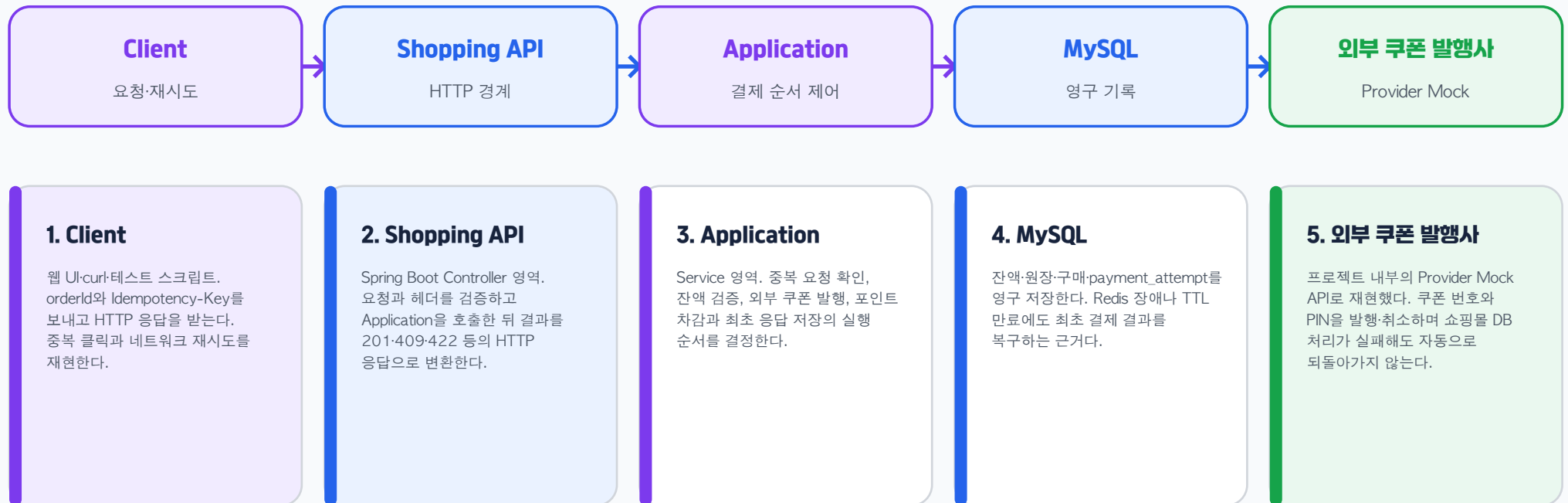
<https://github.com/jsoyun/point-payment-lab>

포인트 결제 시스템을 구성하는 5개의 역할

쇼핑몰 포인트로 외부 바우처를 구매하는 과정을 재현하고, 시스템 경계에서 발생한 중복과 불일치를 개선했다.

프로젝트 목적

Legacy 결제의 외부 중복 발행·Deadlock·잔액 부족 선발행 문제를 재현했다. 이후 같은 요청은 Redis와 payment_attempt, 외부 중복은 외부 발행사의 orderId 중복 방지, 잔액 경쟁은 조건부 UPDATE가 담당하도록 요청 흐름을 단계적으로 바꿨다.



Shopping API와 Application은 같은 Spring Boot 서버의 계층이며, 외부 쿠폰 발행사는 Provider Mock API로 재현했다.

기술 스택과 단계별 구현: 중복 실행부터 잔액 경쟁까지

각 기술을 도입한 이유와 API 흐름에서 담당하는 책임을 단계별로 연결했다.

Backend

Java 17 · Spring Boot · Spring Data JPA · TransactionTemplate로 API와 결제 트랜잭션을 구현했다.

Data & Migration

MySQL 8 · Flyway로 잔액·원장·구매·결제 시도 스키마와 변경 이력을 관리했다.

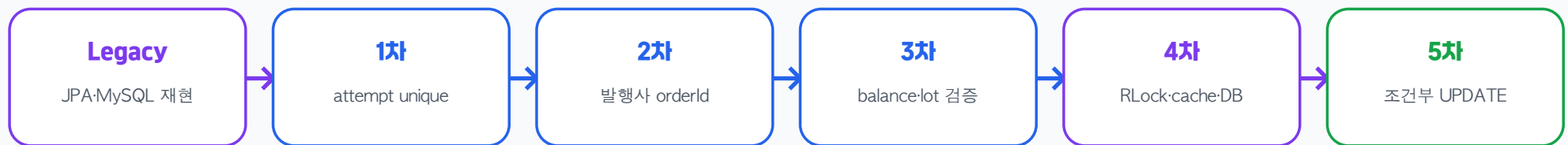
Concurrency

DB unique · saveAndFlush · Redisson RLock · 조건부 UPDATE로 경쟁 지점을 분리해 방어했다.

Infra & Verification

Docker Compose · Redis · curl 병렬 스크립트로 두 인스턴스·장애·동시 요청을 검증했다.

단계별 구현과 기술적 변화

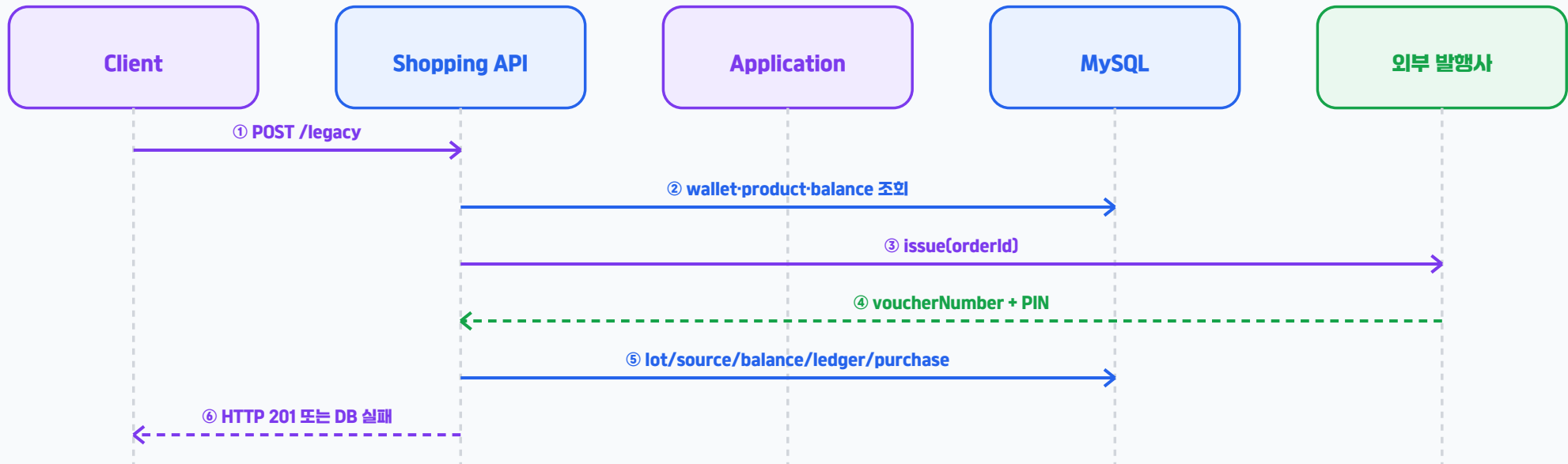


Redis를 사용한 이유

여러 Spring Boot 인스턴스가 같은 요청의 실행 상태와 완료 결과를 공유하기 위해 Redisson 분산락과 Redis cache를 추가했다. MySQL에는 최초 응답을 남겨 Redis 장애에도 결제를 재실행하지 않는다.

Legacy: 외부 HTTP 호출과 MySQL 트랜잭션의 불일치

내부 결제 트랜잭션이 rollback되더라도, 외부 쿠폰 발행사에서 이미 발급한 쿠폰은 자동으로 취소되지 않는다.



기존 문제점

외부 쿠폰 발행이 포인트 차감보다 먼저 실행된다. 이후 쇼핑물 DB 처리가 실패해도 이미 발급된 쿠폰은 자동으로 취소되지 않는다.

수정한 부분

이 페이지는 수정 전 비교 기준이다. 이후 개선에서 요청 순서가 어떻게 달라졌는지 비교하기 위해 Legacy API를 그대로 보존했다.

개선된 결과

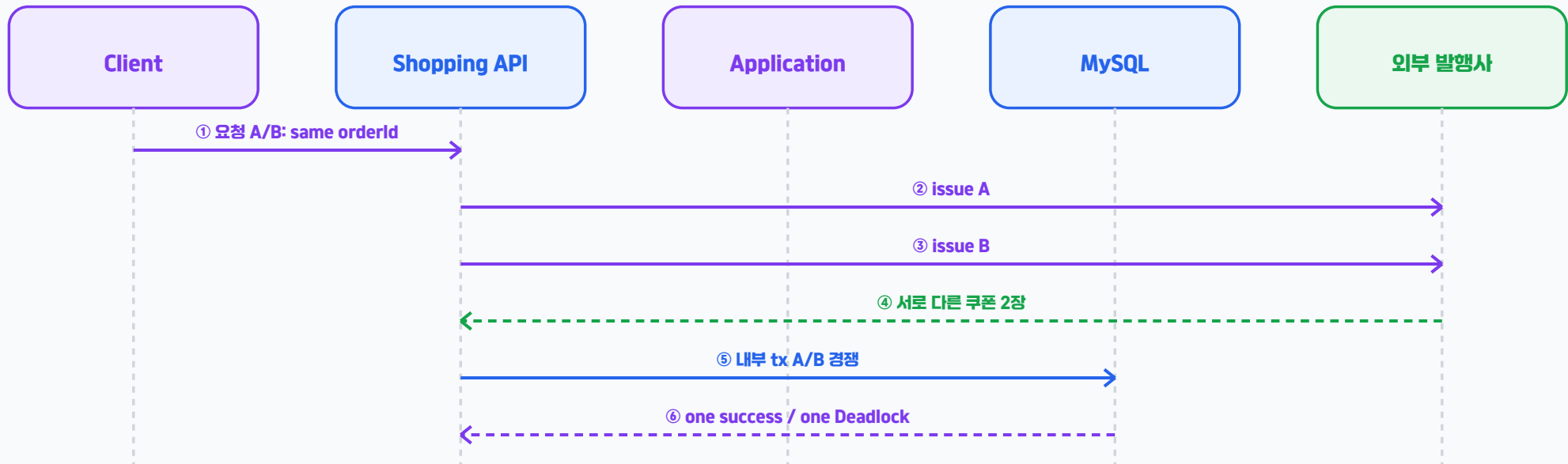
정상 결제는 HTTP 201을 반환하지만, 내부 결제가 실패하면 외부 쿠폰 취소 API를 별도로 호출해야 한다.

달라진 이유

쇼핑물 MySQL 트랜잭션과 외부 쿠폰 발행 API는 하나의 트랜잭션으로 묶여 있지 않기 때문이다.

문제 재현: 동일 orderId 동시 요청으로 쿠폰 2장·Deadlock

쇼핑몰 구매 기록이 1건이어도 외부 쿠폰은 2장 발급될 수 있음을 HTTP 응답과 DB 기록으로 확인했다.



기존 문제점

두 요청 중 하나를 먼저 선택하는 장치가 없어 둘 다 외부 쿠폰 발행 API를 호출했다. 이후 서로 상대가 가진 DB lock을 기다리면서 Deadlock도 발생했다.

수정한 부분

수정 전 결과를 남기기 위해 두 HTTP 응답과 외부 쿠폰·쇼핑몰 구매·잔액·포인트 묶음의 DB 기록을 함께 수집했다.

개선된 결과

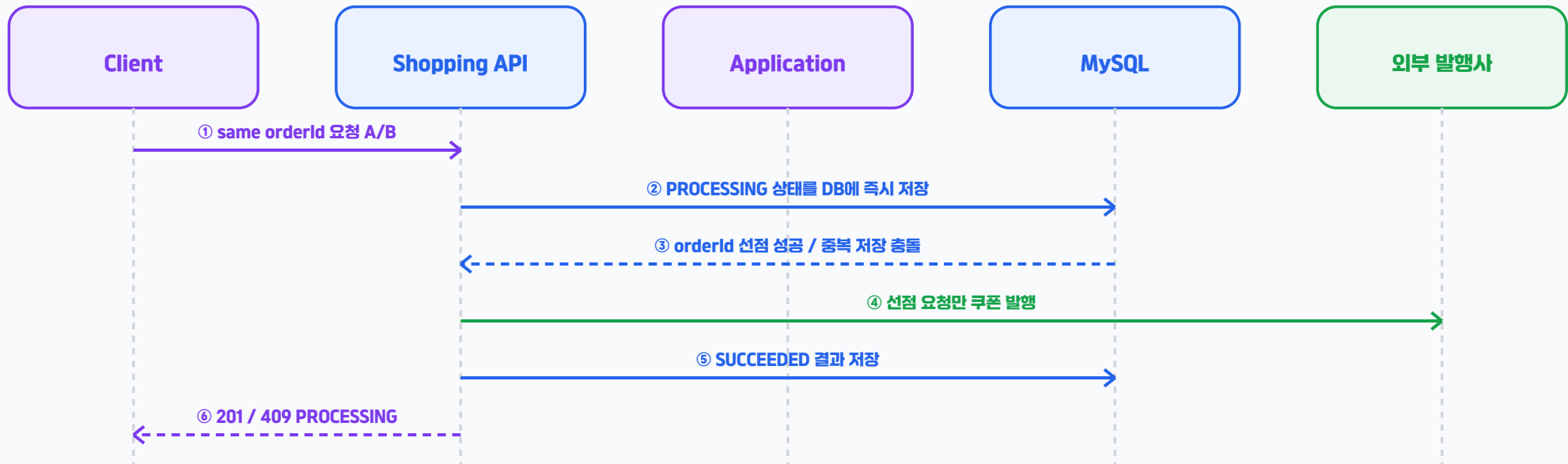
한 요청은 Deadlock으로 HTTP 500, 다른 요청은 HTTP 201을 받았다. 외부에는 발급 1건과 취소 1건, 쇼핑몰 구매는 1건이 남았다.

달라진 이유

쇼핑몰 DB의 중복 방지는 최종 구매 저장만 막았을 뿐, 두 요청이 외부 쿠폰을 발행하기 전에 하나만 선택하지 못했기 때문이다.

1차: payment_attempt unique + saveAndFlush로 실행권 선점

같은 orderId를 payment_attempt에 먼저 저장한 요청만 실제 결제를 실행한다.



기존 문제점

외부 호출 전에 주문 처리 상태를 기록하지 않아 두 요청이 동시에 쿠폰 발행사로 전달됐다.

수정한 부분

orderId unique로 한 요청만 저장되게 했다. saveAndFlush는 INSERT SQL을 즉시 실행해, 외부 쿠폰 발행 전에 선점 성공 또는 중복 충돌을 확인한다.

개선된 결과

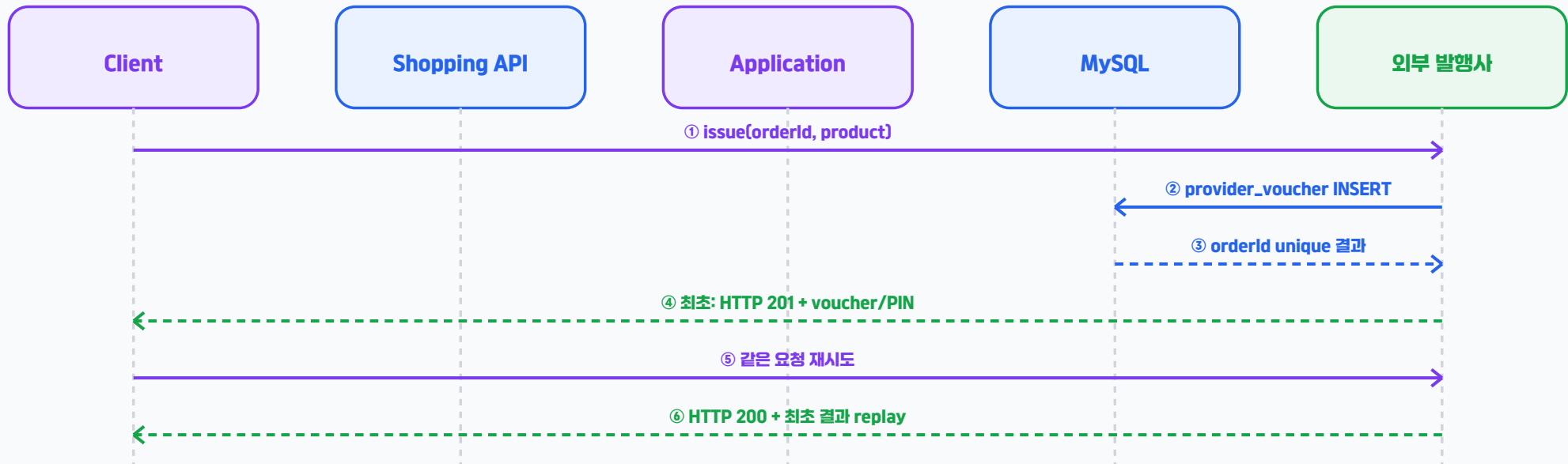
외부 쿠폰 발행은 2건에서 1건, Deadlock과 보상 취소는 각각 1건에서 0건으로 줄었다. 두 번째 요청은 HTTP 409로 종료된다.

달라진 이유

save만 쓰면 INSERT가 늦어질 수 있다. saveAndFlush로 제약 확인 시점을 외부 호출보다 앞당겼다. flush는 commit이 아니라 SQL 실행 시점만 앞당긴다.

2차: provider_voucher orderId unique로 외부 발행 역등성 보장

쇼핑몰 경로를 우회해 외부 발행사 API를 직접 호출해도 쿠폰은 한 장만 생성된다.



기존 문제점

쇼핑몰 payment_attempt는 외부 발행사 API를 직접 호출하는 요청까지 보호하지 못했다.

수정한 부분

외부 발행사 DB에도 orderId를 중복 저장할 수 없게 했다. 같은 상품의 재요청에는 최초 쿠폰을 반환하고, 다른 상품에는 HTTP 409를 반환한다.

개선된 결과

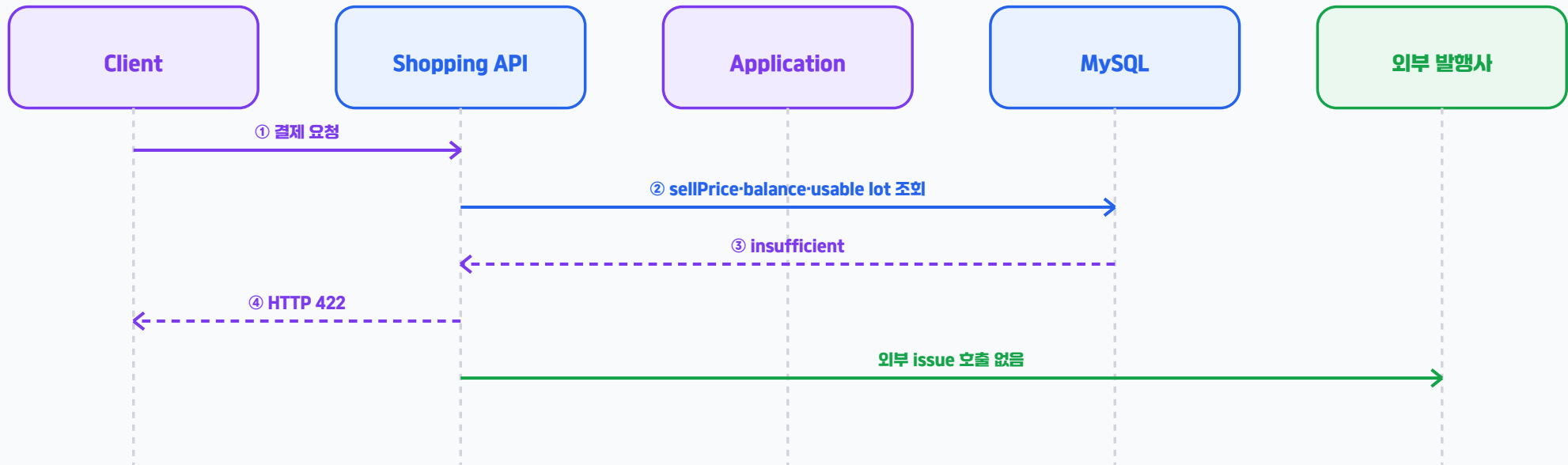
두 발행 요청이 동시에 들어와도 두 응답의 쿠폰 번호와 PIN이 같고, 외부 발행사 DB에는 쿠폰이 1건만 저장된다.

달라진 이유

쇼핑몰을 우회한 요청도 외부 발행사 자신의 DB에서 같은 orderId를 한 번만 저장하도록 보장했기 때문이다.

3차: 외부 호출 전 point_balance·point_lot 사전 검증

결제할 수 없는 요청에는 쿠폰 발행과 취소 API를 아예 호출하지 않는다.



기존 문제점

포인트가 부족해도 쿠폰을 먼저 발행했기 때문에 내부 결제 실패 후 다시 쿠폰 취소 API를 호출해야 했다.

수정한 부분

외부 쿠폰 발행 전에 상품 가격, 총 포인트 잔액, 만료되지 않고 사용하지 않은 포인트 묶음의 합계를 검증했다.

개선된 결과

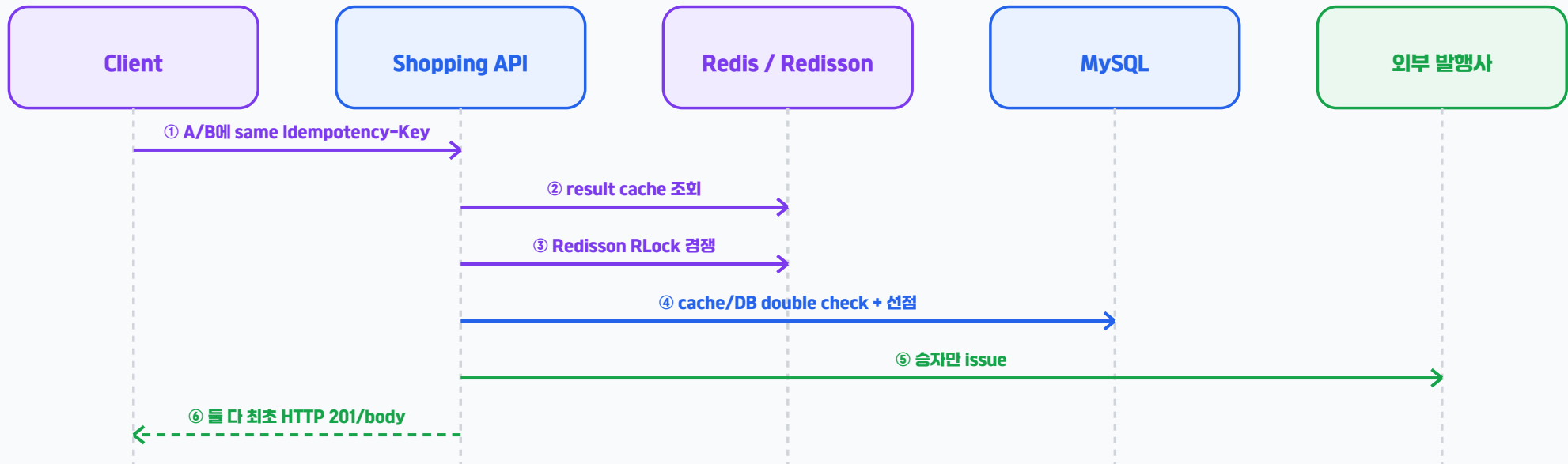
잔액 부족 응답은 HTTP 500에서 422로 바뀌고, 외부 쿠폰 발행과 취소 호출 및 내부 데이터 변경은 모두 0건이 됐다.

달라진 이유

쇼핑몰이 미리 판단할 수 있는 결제 실패 조건을 외부 API 호출 전에 모두 확인하도록 순서를 바꿨기 때문이다.

4차: Redisson RLock + Redis cache + MySQL 최초 응답 저장

Redis는 같은 요청이 동시에 실행되지 않게 조율하고, MySQL은 장애 후에도 최초 결제 결과를 복구할 수 있게 보존한다.



기존 문제점

서버가 두 대이면 각 서버의 Java 메모리를 공유할 수 없다. DB만 사용한 재요청도 매번 중복 INSERT와 결과 SELECT를 수행했다.

수정한 부분

요청 내용의 SHA-256 식별값, 만료 시간이 있는 Redis 결과 cache, Redisson 분산락과 최초 HTTP 응답의 MySQL 저장을 추가했다.

개선된 결과

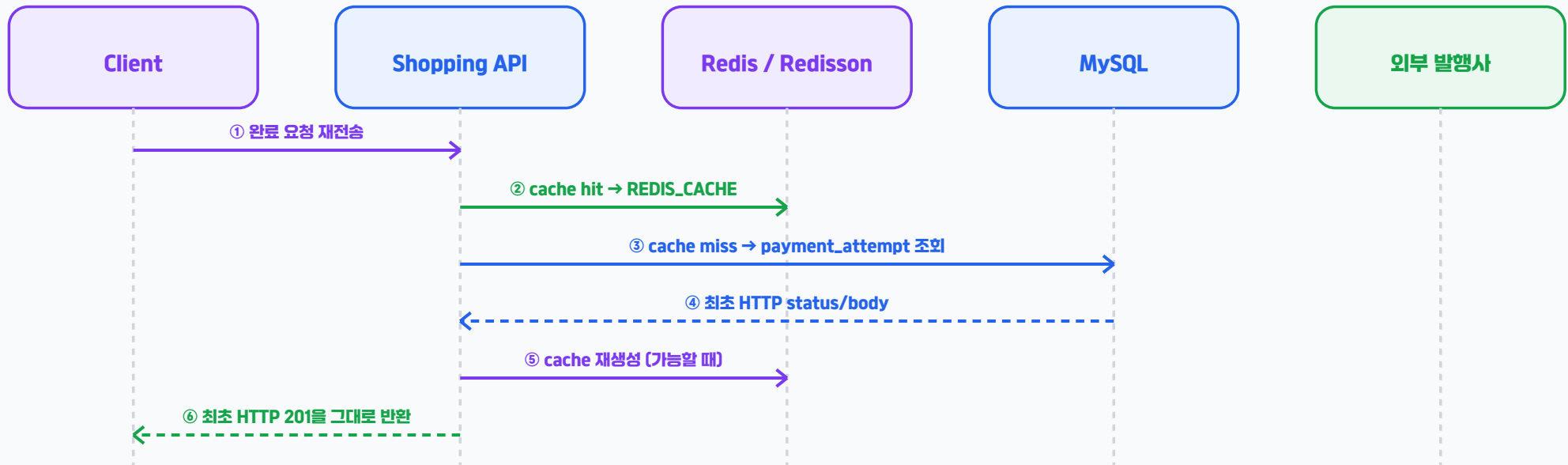
8080과 8081에 동시에 요청해도 외부 쿠폰과 쇼핑물 구매는 각각 1건만 생성되고, 두 요청은 같은 HTTP 201과 응답 본문을 받았다.

달라진 이유

두 서버가 공유하는 Redis lock이 동시 실행을 하나로 줄이고, Redis 정보가 사라져도 MySQL의 최초 응답으로 복구할 수 있기 때문이다.

Redis cache miss-TTL 만료·장애 시 MySQL 결과로 복구

Redis cache가 만료되거나 중단되어도 결제를 다시 실행하지 않고 MySQL에 저장한 최초 응답을 반환한다.



기존 문제점

기존에는 완료된 결제를 다시 요청하면 최초와 다른 HTTP 200과 안내 문구를 반환했고, 매번 MySQL을 조회했다.

수정한 부분

payment_attempt에 최초 HTTP 상태 코드와 응답 본문을 저장했다. Redis를 사용할 수 없으면 MySQL 기록에서 같은 응답을 복구한다.

개선된 결과

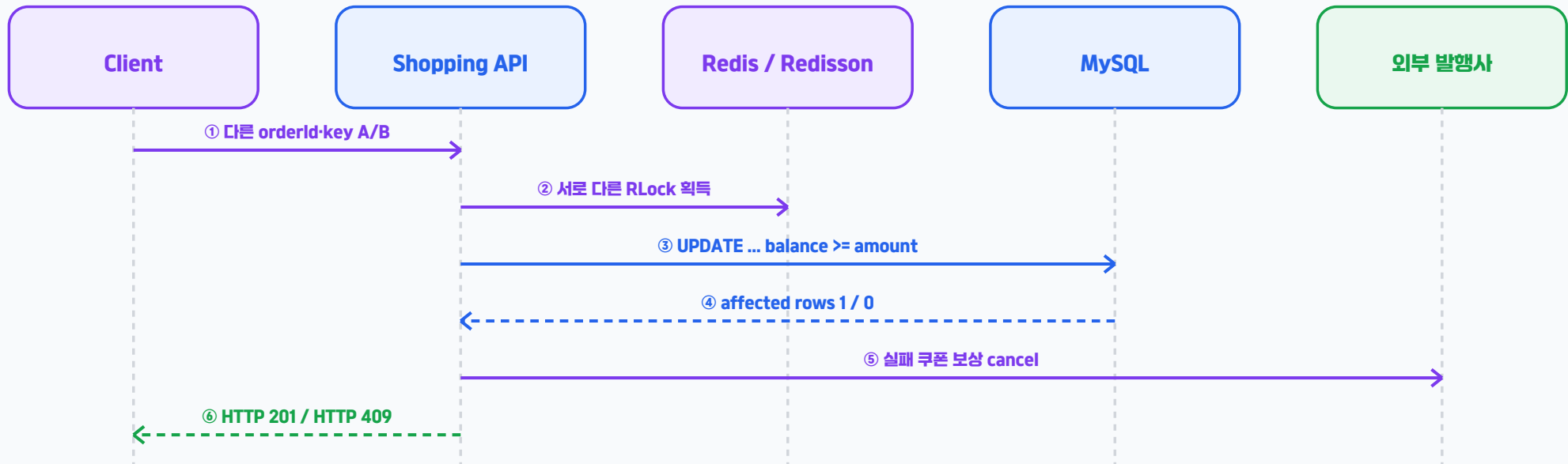
cache 삭제와 Redis 중단 후에도 최초 HTTP 201을 그대로 반환한다. 같은 맥동기로 다른 결제를 보내면 HTTP 422로 거절한다.

달라진 이유

빠른 조회용 Redis뿐 아니라 영구 저장소인 MySQL에도 최초 응답을 보존해, Redis 장애와 정확한 응답 재사용을 분리했기 때문이다.

5차: MySQL 조건부 UPDATE로 서로 다른 주문의 잔액 경쟁 방어

서로 다른 먹등키는 서로 다른 Redis lock을 사용하므로, MySQL이 잔액을 한 번에 확인하고 차감하게 했다.



기존 문제점

서로 다른 두 주문이 동시에 잔액 5,000을 읽으면, 먹등키가 다르기 때문에 각각 결제를 시작할 수 있다.

수정한 부분

잔액이 결제 금액 이상일 때만 차감하는 하나의 UPDATE를 실행하고, 실제 변경된 행이 1건인 요청만 결제를 계속했다.

개선된 결과

한 요청은 HTTP 201, 다른 요청은 잔액 경쟁을 뜻하는 HTTP 409를 받았다. 최종 잔액은 0이고 구매는 1건만 생성됐다.

달라진 이유

MySQL이 잔액 행을 잠금 상태에서 조건을 다시 확인하므로, 먼저 차감한 요청 이후의 최신 잔액을 기준으로 두 번째 요청을 거절하기 때문이다.

개선 전후 HTTP·외부 쿠폰 호출·요청 중단 위치 비교

HTTP 변화는 단순 상태 코드 변경이 아니라 요청이 멈추는 위치가 앞당겨진 결과다.

시나리오	기존	개선 후	외부 쿠폰 발행	멈추는 위치
같은 orderId 동시	201 / 500	201 / 409	2 → 1	orderId 처리권 선점
외부 발행사 직접 중복	201 / 201	201 / 200 최초 결과	2 → 1	발행사 orderId 방어
잔액 부족	500	422	1 → 0	외부 호출 전 검증
같은 멍등키 완료	200 다른 응답	201 최초 응답	추가 0	최초 결과 저장
다른 주문 잔액 경쟁	중복 차감 위험	201 / 409	2*	조건부 잔액 차감

응답이 달라진 공통 이유

실패와 경쟁을 외부 호출보다 앞에서 판단하고, 완료된 요청은 결과를 새로 만들지 않고 최초 HTTP 응답을 그대로 저장해 반환했다.

주의할 결과

서로 다른 주문의 잔액 경쟁은 외부 쿠폰 발행 후 한 요청이 실패한다. 따라서 실패한 쿠폰을 취소한 기록 1건이 남으며, 포인트 예약이 다음 과제다.

* 서로 다른 주문은 둘 다 사전 검증을 통과해 외부 쿠폰을 발행할 수 있다. 조건부 UPDATE는 포인트 중복 차감은 막지만 외부 쿠폰 호출까지 줄이지는 못한다.

최종 방어 구조: Redis lock·DB unique·조건부 UPDATE의 역할 분리

각 방어선은 앞 단계가 실패하거나 우회해도 자신의 데이터 경계를 지킨다.

1

사전 검증

가격·총잔액·lot 부족이면 외부 호출 전 HTTP 422

2

Redis RLock + cache

같은 맥登키의 동시 실행은 하나로 줄이고, 완료 요청에는 최초 응답을 반환

3

payment_attempt 중복 방지

Redis 장애·cache 만료에도 요청 실행 여부와 최초 응답을 MySQL에 보존

4

외부 발행사 orderId 중복 방지

쇼핑몰을 우회해 직접 호출해도 외부 쿠폰은 한 장만 발급

5

조건부 balance UPDATE

서로 다른 주문이 같은 포인트를 중복 사용하지 못하게 원자적 차감

검증 결과: 외부 발행 2→1·Deadlock 1→0·재요청 평균 68% 감소

재현 스크립트와 가공하지 않은 HTTP·DB 결과를 저장해 같은 시나리오를 다시 실행할 수 있다.

외부 중복 발행

2건 → 1건

Deadlock

1건 → 0건

잔액 부족 쿠폰 발행

1건 → 0건

재요청 평균

8.118 ms → 2.588 ms

검증	증거
Legacy 동시 주문	HTTP A/B + provider_voucher + voucher_purchase + balance/lot
Redis 두 인스턴스	201/201 동일 body + provider/purchase/payment_attempt 각 1건
Redis 장애	DATABASE_REPLAY + 최초 HTTP 201/body
조건부 잔액 경쟁	201/409 + balance 0 + purchase 1 + CANCELED 1

성능 수치의 범위

로컬에서 완료 요청을 100회 순차 재전송한 비교다. 운영 성능으로 과장하지 않고, Redis에 저장된 결과를 바로 반환하면 DB 중복 저장 시도와 조회를 피할 수 있다는 근거로 사용했다.

결론: 분산 멍등성·원자적 잔액 차감·외부 보상 처리의 역할 분리

기술 나열이 아니라 API 흐름이 왜 달라졌는지를 설명할 수 있는 프로젝트로 정리했다.

문제에서 배운 점

최종 purchase 한 건만 보면 외부 쿠폰 두 장과 보상 취소를 놓친다. 시스템 경계의 부작용까지 성공 기준에 포함해야 한다.

개선에서 배운 점

멍등성은 중복 요청을 단순히 무시하는 것이 아니다. 요청 내용이 같은지 확인하고 최초 HTTP 상태와 본문을 안전하게 다시 반환하는 것이다.

설계에서 배운 점

Redis 분산락, DB 중복 방지, 조건부 잔액 차감과 외부 쿠폰 취소는 서로 다른 문제를 해결하므로 서로를 대신할 수 없다.

다음 과제

포인트 예약

외부 쿠폰 발행 전 RESERVED 상태로 경쟁 실패분의 불필요한 발행·취소 제거

보상 Outbox

쿠폰 취소 실패 영속화, 지수 backoff, reconciliation과 운영 알람

운영·통합 테스트

Redis HA·TLS·관측성과 Testcontainers 동시성 회귀 테스트

기존 API 흐름 → 문제 지점 → 코드·DB 수정 → 응답·데이터 변화 → 달라진 이유